SOS-DP-Subset-Optimizer

A subset optimization tool using Sum Over Subsets (SOS) Dynamic Programming | Java 17 · Maven · JavaFX | DSA-3 (25CS2103E)

The problem

Given a set of integers and a query mask Q, compute **F(Q)** = the sum of subset-sums of *every submask* of Q. Example: Q = {1,2} → submasks ∅,{1},{2},{1,2} → sums 0+1+2+3 = **F(Q)=6**.

Two methods, same answer, different cost

Method	Per query	All 2^n masks	Space
Brute force	O(2^k)	O(3^n)	–
SOS DP	O(1) after build	O(n·2^n) build	O(2^n)

Measured result (n = 18, all 262,144 masks)

419 ms brute force vs **4 ms** SOS DP → **~100× faster**
Every one of the 2^18 answers verified equal. Steps: 387,420,489 (3^n) vs 4,718,592 (n·2^n).

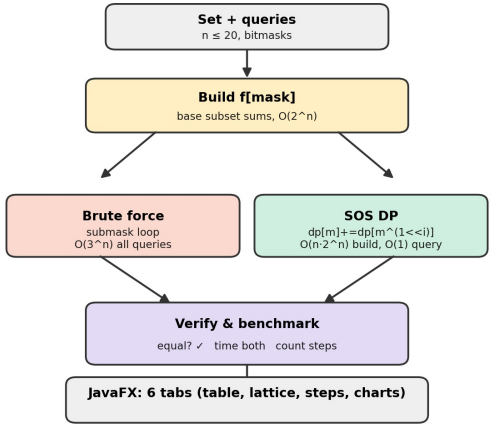
What the app proves

- Builds the base array `f[mask]` once, runs **both** algorithms on it
- Automatically checks every brute-force answer equals the SOS DP answer
- Times both with `System.nanoTime()` and counts steps
- 15 JUnit tests pass; core algorithm uses only arrays — no `java.util`

Key idea (say this out loud)

"*Process one bit at a time over the whole array: if mask has bit i set, `dp[mask] += dp[mask ^ (1<<i)]`. After n bits, every submask has been counted exactly once — turning an exponential re-scan into a one-time O(n·2^n) pass.*"

Program flow



Run it in VS Code

1. Open the project folder in VS Code
2. Terminal: `mvn javafx:run` (Maven fetches JavaFX)
3. Click **Load sample**, then **Run**
4. Backup proof: `mvn test` → 15/15 pass

Sample input to type

Set: `5, 3, 8, 2, 7, 1`

Queries (one per line — three formats):

```
45
0b101101
{0,1,2}
63
0
```

Say this out loud at the start

"*I answer the same subset query two ways, prove they agree, and measure a ~100× speed-up — that's this whole project.*"